2026 56th Annual IEEE International Conference on Dependable Systems and Networks (DSN) | 979-8-3315-5149-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/DSN69566.2026.00050

Reentrancy Detection in the Age of LLMs

1st Dalila Ressi
Ca' Foscari University of Venice
dalila.ressi@unive.it

2nd Alvise Spanò
Ca' Foscari University of Venice
alvise.spano@unive.it

3rd Matteo Rizzo
Ca' Foscari University of Venice
matteo.rizzo@unive.it

4th Lorenzo Benetollo
University of Camerino
lorenzo.benetollo@unicam.it

5th Sabina Rossi
Ca' Foscari University of Venice
sabina.rossi@unive.it

***Abstract*—Reentrancy remains one of the most critical classes of vulnerabilities in Ethereum smart contracts, yet widely used detection tools and datasets continue to reflect outdated patterns and obsolete Solidity versions. This paper adopts a dependability-oriented perspective on reentrancy detection in Solidity 0.8+, assessing how reliably state-of-the-art static analyzers and AI-based techniques operate on modern code by putting them to the test on two fronts. We construct two manually verified benchmarks: an Aggregated Benchmark of 432 real-world contracts, consolidated and relabeled from prior datasets, and a Reentrancy Scenarios Dataset (RSD) of 143 handcrafted minimal working examples designed to isolate and stress-test individual reentrancy patterns. We then evaluate 12 formal-methods-based tools, 10 machine-learning models, and 9 large language models. On the Aggregated Benchmark, traditional tools and ML models achieve up to 0.87 F1, while the best LLMs reach 0.96 in a zero-shot setting. On the RSD, most tools fail on multiple scenarios, the top performer achieving an F1 of 0.76, whereas the strongest model attains 0.82. Overall, our results indicate that leading LLMs outperform the majority of existing detectors, highlighting concerning gaps in the robustness and maintainability of current reentrancy-analysis tools.**

***Index Terms*—Smart Contracts, Reentrancy, Dependability, Static Analysis, Machine Learning, Large Language Models.**

I. Introduction

Reentrancy is the archetypal Ethereum vulnerability. Since the DAO exploit in 2016 [31], where an attacker drained millions of dollars worth of Ether by recursively invoking a withdrawal function, it has remained a central concern for auditors, tool builders, and platform operators. Recent estimates attribute approximately $908.6M in losses to reentrancy-based attacks [65, 19, 48]. Over the years, dozens of detection tools, ranging from symbolic execution engines [41] in conjunction with SMT solvers [38] to abstract interpreters and pattern-based analyzers [57, 32], have been introduced to identify reentrancy flaws before deployment. Additionally, a number of tools perform runtime monitoring [50], fuzzing [45], tracing [67] and other dynamic strategies [34] to detect a variety of vulnerabilities after deployment, including reentrancy.

However, the dependability of this tool ecosystem has degraded over time. Solidity has evolved substantially, especially in the 0.8.x series, introducing new language features, revised error semantics, and modernized contract architectures. Many detectors have not kept pace: some support only legacy compiler versions, others crash on contemporary syntax, and several implement outdated notions of reentrancy tied to the original "checks-effects-interactions" (CEI) formulation [40]. As a result, the long-term reliability of these tools, once viewed as stable components in the smart contract security pipeline, is increasingly uncertain.

This deterioration also affects machine-learning-based detectors. Most ML models are trained on datasets labeled by these same static tools. When detectors disagree, fail to run, or produce inconsistent warnings, these errors propagate directly into the training labels. The result is an ecosystem where ML detectors inherit tool-specific noise, outdated assumptions, and false positives, while little is known about their behavior when evaluated on clean, manually verified data.

The goal of this paper is to quantify the robustness, consistency, and long-term reliability of existing tools and AI-based techniques. We distinguish between *reliability* and *robustness* of reentrancy detectors. Reliability refers to the correctness of predictions, i.e., the ability to accurately detect vulnerabilities while minimizing false positives and false negatives. Robustness denotes the capability of a detector to remain functional across evolving Solidity versions, language features, and contract patterns. Together, these properties determine the overall *dependability* of a detection approach.

Our evaluation relies on two complementary datasets. First, we build an *Aggregated Benchmark* by consolidating and manually verifying contracts from three prior academic datasets whose authors claim manual labeling. Although this benchmark includes both real contracts and injected vulnerabilities, every entry was revalidated through multi-reviewer inspection and LLM-assisted explanation analysis, yielding a substantially higher-confidence alternative to tool-labeled corpora commonly used in ML research. Second, we construct the *Reentrancy Scenarios Dataset* (RSD), a handcrafted suite of minimal contracts (less than 30 lines) covering up-to-date Solidity features and patterns, such as error handling, advanced modifier-based guards, event emissions and code refactorings that obscure syntactic CEI guidelines. RSD is intentionally not representative of real-world prevalence: its purpose is to stress-test detectors under semantic corner cases, evolving language constructs and new habits within the dev community.

These datasets enable an unprecedented evaluation of the

2158-3927/26/$31.00 ©2026 IEEE
DOI 10.1109/DSN69566.2026.00050

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

dependability of 12 existing detection tools in the modern context. *Our findings reveal a fractured and unreliable ecosystem*, with many tools failing outright on modern contracts, disagreeing sharply with one another, or producing high rates of false positives and false negatives even on relatively simple patterns. In addition, we test 10 traditional ML models and 9 LLMs. Several LLMs exhibit strong zero-shot performance across both datasets, achieving high accuracy without version-specific maintenance or handcrafted rules. Their explanations additionally help identify mislabeled edge cases and clarify ambiguous scenarios.

To structure this investigation, we articulate four research questions:

- **RQ1:** How reliable and robust are existing reentrancy detection tools when applied to modern Solidity codebases and evaluated under stress-testing scenarios, given their lack of maintenance and outdated compiler support?
- **RQ2:** How consistent are the warnings and vulnerability reports produced by different tools when applied to the same contracts?
- **RQ3:** Do general-purpose LLMs demonstrate higher dependability and lower maintenance cost compared to static analysis and ML-based detectors?
- **RQ4:** Can LLM-generated explanations assist human experts in identifying mislabeled or ambiguous cases, thus improving dataset quality?

Overall, our results demonstrate that the reliability of conventional reentrancy detectors has significantly eroded as Solidity evolved, calling into question their suitability as ground truth for both developers and ML pipelines. At the same time, LLMs (despite lacking formal guarantees) offer surprising robustness and semantic generality, outperforming outdated static analyzers on both stress tests and manually verified benchmarks [24, 29]. These findings underscore the need for new hybrid approaches that combine the principled guarantees of formal tools with the adaptability of modern AI systems, supported by high-quality human-verified datasets rather than legacy tool outputs.

A. Contributions

- **A dependability-driven assessment of reentrancy detectors.** We provide the first systematic evaluation of the long-term dependability of existing reentrancy detection tools, revealing widespread fragility caused by maintenance decay, outdated compiler support, and inconsistent behavior across detectors.
- **Two complementary datasets enabling reliability and stress-test evaluation.** We release (i) the *Reentrancy Scenarios Dataset (RSD)*, a curated stress-testing suite covering modern and challenging Solidity constructs, and (ii) an *Aggregated Benchmark* built from three manually verified academic datasets. Together, these datasets enable both realistic reliability assessment and controlled robustness evaluation.
- **LLMs as a robust alternative and as human-in-the-loop assistants.** We show that several large language models outperform traditional detectors in accuracy and robustness while requiring no maintenance. Furthermore, their explanations help human auditors identify mislabeled or ambiguous cases, improving dataset quality and annotation reliability.

All code, datasets, and prompts are available in a proprietary repository.[1]

II. Threat Model

We consider Ethereum smart contracts written in Solidity and deployed on an EVM-compatible blockchain. Contracts are compiled to bytecode and interact via message calls that may optionally transfer Ether. In Solidity, external calls can be expressed either through a low-level primitive:

```
target.call{value: amount}(data)
```

where `target` is an `address`, `amount` is an integral expressing the amount of Ether being transferred, and `data` encodes both the callee function name and actual arguments; or via a high-level call:

```
I(target).f(args..)
```

where `I` is an interface type (e.g. ERC20), `f` is a function name and `args..` is the argument list. Both syntaxes are compiled into the same `CALL` bytecode instruction; thus, they are equivalent at the runtime level. A transaction may trigger a chain of contract invocations, including callbacks into the original caller contract - this is what reentrancy is about. The contract state consists of its persistent storage variables and its internal balance. We assume that the underlying blockchain provides standard consensus and integrity guarantees and that EVM opcodes behave as specified.

A. Attacker Capabilities and Goals

The adversary controls one or more externally owned accounts and can deploy arbitrary malicious contracts. The attacker can:

- invoke any public or external function of the victim contract in arbitrary order, with chosen calldata and Ether values;
- implement fallback or `receive` functions that re-enter the victim during an external call;
- create other contracts that can invoke any function of the victim in any order at their construction time.

The attacker *cannot* break cryptographic primitives, bypass the consensus protocol, forge signatures, or alter historical blockchain state. We focus solely on vulnerabilities arising from the contract logic and its interaction patterns.

The adversary's goal is to exploit *reentrancy*: gaining multiple, nested entries into the victim contract within the same transaction so as to violate intended invariants. Typical consequences include draining Ether or tokens, inflating balances, tricking a contract to pay more times than intended, or otherwise reaching a contract state that would be impossible under any non-reentrant execution schedule.

[1]Repository URL: https://github.com/matteo-rizzo/reentrancy-detection-benchmarks

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

B. Operational Definition of Reentrancy

We consider a contract vulnerable to reentrancy if, during the execution of one of its entry points:

1) it performs an external call to attacker-controlled code (e.g., via `call`, high-level interface calls, or `delegatecall`);
2) upon return from that external call, it performs a state update that depends on the attacker's interaction (e.g., modifying storage associated with `msg.sender`);
3) and an attacker can re-enter the contract during the external call and drive it to a final state that *cannot* be reproduced by any sequence of the same calls executed without reentrancy.

Conditions (1)-(2) represent the minimal execution structure that *permits* reentrancy; condition (3) characterizes when such a structure is *exploitable*, i.e., when reentrant execution leads to a semantically different final state. This definition generalizes the classical checks–effects–interactions (CEI) violation pattern: CEI-compliant code is safe by construction, but CEI alone is insufficient to cover subtle vulnerable patterns such as read-only reentrancy, proxy-mediated flows, or semantics that intentionally place a state update after an external call. This definition also highlights a central challenge: detecting reentrancy requires semantic reasoning, yet most existing datasets and tools rely on brittle syntactic rules.

III. Motivation

A central challenge in evaluating reentrancy detectors is that no agreed-upon definition of reentrancy exists in practice. During our preliminary investigation, we found that both academic papers and widely used tools implement incompatible, outdated, and often underspecified heuristics. As a result, the ecosystem lacks a reliable ground truth: detectors disagree systematically, ML datasets inherit contradictory labels [49], and many entries in "manually verified" datasets are not manually verified at all [28].

a) Why manually labeled datasets were our starting point: To avoid circularity in our evaluation (where tools are assessed on data labeled by other tools) we began with the only three datasets that explicitly claim human validation: the Consolidated Ground Truth dataset (CGT) [28], the Huang-Gai dataset [18], and the large-scale reentrancy dataset used in [68]. However, the CGT paper itself documents inconsistencies in prior manual labeling efforts, showing that the same contract is often labeled differently across publications. This forced us to reexamine each dataset, rather than rely on legacy annotations implicitly shaped by outdated detectors.

b) Why we extracted the rules of existing tools: To understand the sources of disagreement, we analyzed how the main reentrancy analyzers actually define the vulnerability. This analysis revealed a fragmented landscape: every tool encodes its own notion of reentrancy, often tied to pre-0.8 compiler behaviors or the 2016 DAO exploit. We selected a group of vulnerability detection tools from the CGT study and integrated it with other peer-reviewed tools as well as tools often used to label smart contracts for the training of AI models.

We briefly summarize the core rule each tool uses, demonstrating how inconsistent the definitions are. Symbolic-execution tools such as **Mythril** [4] and **Conkas** [14] flag reentrancy when a low-level `CALL` forwards more than 2,300 gas and is followed by an `SSTORE`, though Mythril ignores constructor calls and precompiled addresses, whereas Conkas checks SMT-feasible storage dependencies. **Oyente** and Oyente+ [41] treat a call as reentrant when its path condition remains satisfiable after hypothetical recursive execution. Declarative or pattern-based analyzers diverge even more. **CCC** [63] detects reentrancy via Code Property Graph queries requiring a user-influenced path from an external call to a subsequent state update. **Securify** [57] issues violation patterns whenever an `SSTORE` depends on a preceding `CALL`. **Slither** [32] expands the CEI rule into five distinct variants: counting event emissions, unlimited-gas calls, or even certain benign effects as reentrancy. **Solhint** [12] reduces reentrancy to pure CEI violations: any state write after an external call triggers a warning. **Vandal** [22] uses Datalog queries on decompiled bytecode and flags a `CALL` as reentrant when it forwards enough gas, lacks a mutex, and is followed by state updates. Dynamic tools implement yet another definition. **Confuzzius** [56] flags reentrancy only when a concrete execution trace includes a `CALL` with more than 2,300 gas, an `SLOAD` before it, and an `SSTORE` to the same slot after it—an explicit encoding of the DAO exploit. **sFuzz** [45] deploys contracts to a sandbox chain and labels a contract vulnerable only if a synthesized attacker contract successfully reenters during fuzzing.

For some of these tools, the precise rules were not properly documented, and we had to look at the implementation to infer the exact mechanism applied. In other cases, the paper reported the detection of reentrancy, but the rule was not implemented in the provided tool [55]. The complete list of the tools we examined can be seen in Table I.

c) Why this fragmentation matters: These rules are not minor variations, they are mutually incompatible interpretations of the same vulnerability. Tools disagree on whether:

- unlimited-gas high-level calls are equivalent to `call()` [41];
- `delegatecall` should always be treated as reentrant [53];
- read-only reentrancy is a type of reentrancy vulnerability [66];
- mutexes or modifiers constitute adequate guards [32].

Many rules further depend on compiler behaviors that no longer hold in Solidity 0.8+, such as automatic underflow checks, fallback semantics, and storage access patterns. This explains why detectors often break on modern code.

d) Implications for ground truth: Because each dataset in the literature was labeled according to a specific tool's rule (often undocumented, especially in papers describing methods to detect multiple vulnerabilities) the resulting labels are inconsistent [28, 49, 30]. This is why prior work frequently

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Comparison of analyzers for reentrancy detection.

Tool	Version/Commit	Working	Input	Techniques	Paper	Repo
Aderyn	v0.6.5	✓	Source	Abstract syntax tree traversal and rule-based static pattern matching	N/A	[6]
AutoAR	N/A	N/A	Mixed	Program dependency graph construction and inter-procedural static analysis	[35]	-
CCC	#c531ae3	✓	Bytecode	Code property graph analysis with Cypher query patterns	[63]	[10]
ConFuzzius	#4315fb7 v0.0.1	✓	Runtime	Hybrid approach combining symbolic execution, syntax inspection, SMT solving, and fuzzing	[56]	[2]
Conkas	#4e0f256	✓	Runtime	Symbolic execution with syntactic and SMT-based constraint analysis	[59]	[13]
ContractWard	N/A	✓	Bytecode	Machine-learning-based vulnerability classification with AST feature extraction	[61]	-
eThor	2023	×	Bytecode	Abstract interpretation, Horn clause resolution, and SMT solving	[53]	-
DefectChecker	N/A	✓	Bytecode	Abstract syntax tree analysis with supervised machine learning	[25]	-
Manticore	v0.3.7	×	Bytecode	Symbolic execution and SMT-based path exploration	[44]	[16]
Mythril	v0.24.8	✓	Bytecode	Symbolic execution with constraint solving and pattern-based vulnerability detection	-	[3]
NPChecker	N/A	N/A	Bytecode	Inter-procedural static analysis with constraint solving	[60]	-
Oyente+	#060ca34	✓	Bytecode	Symbolic execution and SMT constraint analysis	[42]	[15]
Sailfish	v0.1	✓	Bytecode	Symbolic execution combined with static dependency and control-flow analysis	[21]	-
Securify	v1.0	✓	Bytecode	Datalog-based data-flow and call-dependency analysis	[57]	[8]
Securify2	N/A	×	Bytecode	Declarative intermediate-representation analysis using Datalog compliance and violation rules	-	[9]
Sereum	v1.0	N/A	Runtime	Dynamic taint tracking on Ethereum bytecode during runtime execution	[50]	-
sFuzz	#48934c0	✓	Runtime	Feedback-guided fuzzing with transaction sequence generation	[45]	[7]
Slither	v0.11.3	✓	Source	Static pattern matching with inter-procedural data-flow and control-flow analysis	[32]	[5]
Smartcheck	N/A	×	Source	Abstract syntax tree pattern matching and heuristic regular expressions	[55]	-
Solhint	v6.0.0	✓	Bytecode	Linting rules and static pattern matching on Solidity source code	-	[12]
teEther	#04adf56	×	Bytecode	Symbolic execution and SMT-based exploit generation	[39]	[11]
TotalSol	#04adf56	N/A	Mixed	Multi-layer analysis combining control-flow, data-flow and inter-procedural dependency	[43]	-
Vandal	#d2b0043	✓	Bytecode	Bytecode decompilation and Datalog-based static analysis	[22]	[17]

disagrees on whether a contract is vulnerable. Using such datasets for evaluation or ML training introduces tool-induced bias and artifacts [49].

This fragmentation motivated our approach: (1) extract and analyze the actual rules that tools encode; (2) design a unified three-step labeling procedure grounded in the execution model; (3) re-verify all contracts in existing manually labeled datasets; (4) create a second dataset (RSD) to systematically exercise scenarios that existing detector rules mishandle.

Both datasets, as described next, are thus explicitly aligned with our threat model and eliminate the tool-defined inconsistencies that have undermined prior evaluations.

IV. Dataset Construction

Because existing datasets often embed tool-induced inconsistencies [28, 49, 30], we construct two new datasets aligned with the threat model above. One captures semantic stress tests, the other provides high-confidence labels for real-world contracts. Below, we describe their construction and the unified labeling procedure that ensures consistency across both.

A. Aggregated Benchmark

To evaluate detectors on contracts that developers are likely to encounter in practice, we first assemble a high-confidence benchmark from the largest three prior datasets that explicitly claim manual validation: the Consolidated Ground Truth (CGT) [28], HuangGai [18], and the reentrancy study by [68]. These sources were selected because they contain human-labeled reentrancy cases rather than tool-generated labels. In particular, CGT is itself an aggregated dataset, comprising 13 different datasets. Among these, some of the most well-known are ZEUS [38], SolidiFI [36], SmartBugs Curated [30] and SWC registry [2].

[2]https://swcregistry.io/

We aggregate all contracts from these datasets, deduplicate them, and filter out those that fail to compile under standard Solidity compiler versions (most of the contracts are written in Solidity v0.4.x or v0.5.x, with only a few in v0.6.x). Some contracts do not compile due to a missing pragma statement in the preamble; others do so due to random errors in the code, despite the exact compiler version being adopted.

This yields a pool of 73 579 unique, compilable contracts, consisting of 145 potentially reentrant and 73 434 potentially non-reentrant instances, summarized in Table II. As documented in [28], even "manually labeled" corpora often contain contradictory or inconsistent annotations, motivating the need for re-verification under a consistent threat model.

Manual relabeling under our operational definition. All contracts were manually inspected independently by three authors in multiple rounds, which eventually led to the definition of the procedure in Section IV-C. This procedure operationalizes reentrancy through a combination of external-call detection, post-call side effects, and semantic state divergence. A final round was then performed to ensure adherence to the converged procedure. Authors brought complementary expertise: one in programming languages and formal methods for reentrancy detection, one in smart contract development for Ethereum, and one in vulnerability detection and AI-based contract analysis. Disagreements stemmed from occasional misapplications of our reentrancy-labeling procedure (cf. Section IV-C), and were resolved through joint inspection of the more subtle cases.

To enrich the dataset, we selected a subset of contracts from the 73 434 potentially non-reentrant contracts considered safe in [68]. The criteria used by the authors were that not all the tools they used to detect reentrancy on those contracts (Oyente, Mythril, Securify, Smartian [26], Sailfish) did not report the vulnerability. We used their results to select contracts considered safe by at least 4 of those tools,

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

then extracted a subset of this large batch with a sampling strategy that balances coverage with feasibility, while ensuring inclusion of patterns known to confuse analyzers (e.g., modifiers, inheritance, inline effects) by using regular expressions. The threshold of 4 agreeing tools was selected as a trade-off between label confidence and contract diversity. Requiring 5 tools produced an overly homogeneous subset dominated by low-level call patterns, while four preserved broader interaction types (including high-level calls) without significantly weakening consensus. A threshold of 3 tools was not adopted, as it increased the dataset size at the cost of reduced confidence in the ground-truth labels. After all rounds of manual inspection, numerous contracts originally labeled as reentrant were moved to the safe group, and vice versa.

LLM-assisted verification. To further increase confidence, we queried several LLMs (gpt-4o, gpt-4.1, `o3-mini`, `o4-mini`) using a simple prompt requesting a label and explanation. Whenever the output label of an LLM disagreed with our current label annotation, the same three authors reviewed its explanation. In most cases, disagreements were hallucinations; in a minority, the LLM highlighted subtle behaviors we had overlooked. These cases were re-examined manually and relabeled if necessary. LLMs, therefore, did not label the dataset but served as an additional signal to identify edge cases.

Final benchmark. After this two-step verification (manual re-annotation plus LLM-assisted conflict review) the Aggregated Benchmark contains 432 high-confidence contracts: 122 reentrant and 314 safe. During the process, 28 of the 145 potentially reentrant contracts were relabeled safe, while 5 of the 291 sampled potentially safe contracts were relabeled reentrant. The resulting benchmark offers far higher label reliability than any of the original sources and reflects the operational definition established in our threat model.

B. Reentrancy Scenarios Dataset (RSD)

Our analysis of the Aggregated Benchmark, combined with observations of tool behavior and LLM explanations, revealed recurring blind spots in existing datasets. These include limited coverage of modern Solidity 0.8.x features, insufficient representation of alternative external call mechanisms, and a near-complete absence of structurally diverse variants. To address these gaps, we constructed the Reentrancy Scenarios Dataset (RSD), a suite of 143 manually verified Minimal Working Examples (MWEs),

TABLE II: Aggregated Benchmark Dataset Composition.

Contracts	Initial Raw Count	Unique Contracts	Compilable Contracts
# Reentrant in CGT [28]	300	296	93
# Reentrant in HuangGai [18]	22	22	21
# Reentrant in Reentrancy Study [68]	46	40	31
Total **Reentrant** (**before** manual verification)	368	358	145
Total **Reentrant** (**after** manual verification)	145	120	**120**
# Non-Reentrant in Reentrancy Study [68]	123,169	118,422	73,434
Total **Non-Reentrant** (**before** manual verification)	123,169	118,422	73,434
Total **Non-Reentrant** (**after** manual verification)	287	312	**312**
Total in Aggregated Benchmark		432	**432**

The structure of RSD can be seen in Table III. RSD is organized into scenario families (identified by the same "code") reflecting well-known reentrancy patterns and real-world contracts. Each category isolates a distinct source of potential reentrancy, such as breakages of the CEI pattern, misuse of guard mechanisms such as `nonReentrant` and `onlyOwner`, ERC20 token logic, proxy-based delegation, dangerous primitives such as `delegatecall` and so on. For each scenario, we construct multiple variants by applying controlled syntactic and structural transformations or by simply injecting certain constructs. Variants include function folding (`Fold`), error raising (`Error`), event emission (`Emit`), use of modifiers (`Mod`), contract inheritance (`Inherit`), inlining of side effects within a statement (`Inline`), use of `unchecked` blocks (`Unchecked`), use of low-level static call (`Staticcall`), and use of in-place subtraction operator (`Underflow`). These variants do not fabricate unrealistic behaviors; rather, they preserve the original reentrant control flow and EVM semantics while representing syntactic and structural variations commonly found in real-world contracts, precisely targeting patterns that challenge both static analyzers and LLMs.

Table III summarizes the RSD by category, including the number of safe and vulnerable instances per scenario family, the reentrancy type involved, a short description, and the main variants explored. Each category is identified by a code number and a name prefix. We now showcase code examples only for the most representative scenarios, and refer the reader to the full dataset included in the supplementary material.

Basic (00). The simplest scenario features one function performing an external call that can reenter into itself[3].

```
mapping (address => uint256) public bal;

function withdraw(uint256 amt) public {
  require(bal[msg.sender] >= amt, "No funds");
  (bool success, ) = msg.sender.call{value:amt}("");
  require(success, "Call failed");
  bal[msg.sender] -= amt;
}
```

The state update after the external call violates the CEI practice and leads to potential attacks.

SingleMod (03). To programmatically prevent reentrancy, contracts may implement a mutex-like mechanism using a global boolean flag. This flag is typically tested and set at the beginning of the code section that performs an external call, and reset in the epilogue. Unlike traditional mutexes in concurrent systems, atomicity is not required here, as the Ethereum execution model is inherently sequential. Using a mutex to guard reentrant functions enables developers to intentionally deviate from the CEI pattern, safely performing state updates *after* external calls without exposing the contract to reentrancy vulnerabilities. This pattern can be implemented

[3]The state variable `bal` is defined once here and referenced from most snippets in this section.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Scenario taxonomy of the Reentrancy Scenarios Dataset (RSD). Categories group MWEs exhibiting similar structural patterns.

Code	Name Prefix	#Safe	#Ree	Reentrancy Type	Description	Variants
00	Basic	8	7	single function	Basic DAO in multiple variants and flavours	Emit, Error, Fold, Staticcall, Unchecked, Inline
00	BasicConst	1	1	single function	Basic DAO with constant address as target	
00	BasicCross	0	1	cross function	Basic DAO in a cross-function setting	
00	BasicNoChecks	1	1	single function	Basic DAO implementing CEI and omitting the checks part	
00	BasicNoCall	1	0	single function	Dummy contract without external calls	
01	SingleMutex	4	6	single function	DAO-like with flag-based mutex protection in a single function settings	Fold, Underflow
02	CrossMutex	4	4	cross function	DAO-like with flag-based mutex protection in a cross-function setting	Underflow, Unchecked
03	SingleMod	5	8	single function	DAO-like with a modifier guard in a single function settings	Fold, Underflow
04	CrossMod	5	8	cross function	DAO-like with a modifier guard in a cross-function setting	Fold, Underflow
05	Send	6	0	single function	DAO-like using send primitive (always safe)	Emit, Unchecked
06	Transfer	4	0	single function	DAO-like using transfer primitive (always safe)	Unchecked
07	MixedSend	3	3	single function	Send primitive followed by a low-level call	Fold, Emit
08	MixedTransfer	2	2	single function	Transfer primitive followed by a low-level call	Emit
09	ERC20	7	10	single function	Donations via ERC20 tokens mixing various high-level calls	Mod, Inheritance, Staticcall
09	ERC20OnlyOnce	2	1	single function	Donations via ERC20 tokens that can occur only once per user	
09	ERC20Staking	3	3	single function	Staking application with ERC20 tokens	
09	ERC20StakingPull	3	3	single function	Staking application with ERC20 tokens including withdrawal	Mod
10	OnlyOnce	2	0	single function	DAO-like where withdrawal can occur only once per user (always safe)	
11	Proxy	3	0	none	Proxy contract forwarding calls to a given target	Staticcall
12	OnlyOwner	1	1	cross function	Special guard checking if the sender is the original owner of the contract	
13	Loop	1	1	single function	Loop containing CEI-breaking code	
13	LoopCrossMod	3	4	cross function	Loop containing CEI-breaking code in a cross-function setting	
13	LoopCrossMutex	3	4	cross function	Loop guarded by mutexes	
14	DelegateCall	0	4	delegatecall	Delegate calls are always vulnerable under our definition	
15	ReadOnly	3	3	cross contract	Modifications in the state of one contract make another vulnerable	Staticcall

in a couple of equivalent ways - either manually, or through a Solidity modifier that implements a reentrancy guard:

```
modifier nonReentrant() {
  require(!flag, "Locked");
  flag = true;
  _;   // placeholder for the function body
  flag = false;
}
function withdraw(uint256 amt) nonReentrant public {
  require(bal[msg.sender] >= amt, "No funds");
  (bool success, ) = msg.sender.call{value:amt}("");
  require(success, "Call failed");
  bal[msg.sender] -= amt; // effect after call is ok
}
```

Although the snippet above illustrates a non-reentrant sample, flawed implementations of this pattern may still introduce reentrancy vulnerabilities, such as wrongly implementing the boolean flag management or forgetting the require statement.

CrossMod (04). This category includes contracts composed of multiple functions, where one performs an exploitable external call. Despite the reentrancy guard, an attacker can still re-enter other public functions:

```
function transfer(address to, uint256 amt) public {
  require(bal[msg.sender] >= amt, "No funds");
  bal[to] += amt; // reentering here alters state
  bal[msg.sender] -= amt;
}

function withdraw(uint256 amt) nonReentrant public {
  require(bal[msg.sender] >= amt, "No funds");
  (bool success, ) = msg.sender.call{value:amt}("");
  require(success, "Call failed");
  bal[msg.sender] -= amt;
}
```

Such scenarios may be exploited in subtler ways compared to direct single-function reentrancy: the reentrant manipulation of the contract state can result in unintended behaviors, such as altered payment amounts, even when the attack does not involve direct theft of funds.

Send/Transfer (05/06). Solidity provides the `send` and `transfer` primitives as methods of the `address` built-in type. These must not be confused with other `send` or `transfer` methods belonging to interfaces (e.g. ERC20) or third-party libraries. The two primitives differ only for their signature: `send` returns a boolean indicating whether the operation has succeeded, while `transfer` simply reverts the transaction automatically when it fails.

```
function withdraw(uint256 amt) public {
  require(bal[msg.sender] >= amt, "No funds");
  bool success = payable(msg.sender).send(amt);
  require(success, "Call failed");
  bal[msg.sender] -= amt; // effect after call is ok
}
```

Both primitives, however, do *not* allow reentrancy despite invoking the receive/fallback function on the target address (if non-EOA), thanks to a hard limit in the gas at the avail of the target (2300 at the time of writing). Such a small amount does not provide a malicious callee with enough gas to re-enter.

ERC20 (09). The ERC20 standard for managing fungible tokens on Ethereum is defined as a well-known interface in Solidity, which specifies a set of functions and events [1]. Contracts implementing this interface typically entail explicitly type-casting an `address` to the `ERC20` interface and then invoking its methods.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

```
mapping (address => bool) private donated;

function donate(address tok, address to, uint256 amt) public {
  require(!donated[msg.sender]);
  require(IERC20(tok).balanceOf(msg.sender) >= amt * 2, "Need
      twice");
  bool success = IERC20(tok).transfer(to, amt);
  require(success, "Donation failed");
  donated[msg.sender] = true; // side-effect
}
```

Invoking `balanceOf` and `transfer` delegates execution to an external contract whose behavior is unknown, as it may re-enter the caller function or perform other malicious operations. In general, scenarios based on ERC or any other interface can be reduced to the same case: an explicit type-cast from an `address` to a contract or interface type, when followed by a method invocation, is considered a **high-level call**. And that is equivalent to a low-level call under the following reversible syntactic transformation:

$$e_0.\,\textbf{call}\,(\textbf{abi}.\text{encodeWithSignature}(\text{“}\texttt{f}(\tau_1,..,\tau_n)\text{”},e_1,..,e_n))$$
$$\updownarrow$$
$$\texttt{I}(e_0).\texttt{f}(e_1,..,e_n)$$

where expressions $e_0 : \texttt{address}$, $e_i : \tau_i$ for all $i \in [1,n]$ and `I` is an interface type name defining the function signature $\texttt{f}(\tau_1,..,\tau_n)$. The two syntaxes make the Solidity compiler emit the same `CALL` instruction, hence they are equivalent from the point of view of the bytecode. For analyzers processing Soldity code, though, they appear different.

OnlyOwner (12). Another commonly used reentrancy guard is the **`onlyOwner`** modifier, which restricts access to specific functions by ensuring that the caller is the contract's deployer (also known as the owner).

```
modifier onlyOwner() {
  require(msg.sender == owner, "Not authorized");
  _;
}
```

The modifier is often used in place of **`nonReentrant`**, although it just enforces an access control policy. To truly avoid reentrant attacks, especially in cross-function scenarios, a mutex shared among all public/external functions is required.

C. Labeling Procedure

Both the Aggregated Benchmark and the RSD have been labeled according to a unified three-step procedure, not intended as a full formalization of reentrancy but as a coherent operational definition ensuring consistency across datasets. The first two steps rely on transformations and detection patterns that are syntactic in nature, aided by a slight amount of type information. The third step introduces the notion of contract state and ventures into the territory of semantics, refining the detection and ruling out some false positives.

Intuitively, given a Solidity contract, we begin by unfolding local functions (Step 1). We then search in the unfolded code for external calls followed by side-effects (Step 2). If none are found, the contract is classified as safe; otherwise, a *candidate* reentrancy is identified and subjected to an additional semantic analysis (Step 3). This final step compares the contract state produced by executing the nested reentrant calls with the state obtained if the same calls were executed as independent transactions in sequence. In other words, we contrast the outcome of mutually recursive reentrant executions with the outcome of their flattened, sequential counterparts. If the two outcomes coincide, the contract is deemed safe (w.r.t. reentrancy); otherwise, it is vulnerable to reentrancy.

Compared to prior systems, our procedure captures both traditional CEI-breaking patterns and advanced cases (e.g., read-only and cross-contract) under a unified criterion. It diverges from traditional approaches [41, 19, 48] by systematically flagging delegatecalls as exploitable, while excluding emit and error from effects. Also, a flat normalization of modifiers, local calls, and inheritance reduces constructs to a small core, avoiding ad hoc rules and limiting subtle false negatives.

Step 1. Unfolding. The first step performs a semantic-preserving transformation of a Solidity contract through the unfolding of local function calls and of a few other language constructs. Local function calls exhibit no object expression on the left-hand side of the dot and have syntactic form $\texttt{f}(e_1,...,e_n)$ where $n \geq 0$, e_1 .. e_n are arguments of any type, and `f` is a function name identifier. Notable exceptions are calls of the form $\texttt{this.f}(e_1,...,e_n)$, which the Solidity compiler treats as external calls and are therefore not unfoldable.

Function unfolding is a well-studied program transformation technique [23, 58, 47] whereby a function call is systematically replaced with the body of its definition, with actual parameters substituted for formal ones. Given a function $f(x_1 : \tau_1,..,x_n : \tau_n)\ \mathcal{S}$, with $\mathcal{S} = \{S_1;...;S_m\}$ being the function body, where $n \geq 0$, $m \geq 1$, x_1 .. x_n are parameters annotated with their types τ_1 .. τ_n and $S_1..S_m$ are statements, by unfolding each call expression in $\mathcal{S}$ we obtain a function $f'(x_1 : \tau_1,..,x_n : \tau_n)\ \mathcal{S}'$ with $\mathcal{S}' = \{S'_1;...;S'_k\}$, with $k \geq m$. This process is repeated for each *entry-point* in the input contract, i.e. functions marked with **`external`** or **`public`**, as well as constructors, until the whole input contract consists of unfolded functions.

Unfolding also applies to the following language constructs, requiring just a trivial substitution mechanism:

- *Inheritance*: each member (state variable or function) defined in a superclass (parent contract) is embedded in the subclass (derived contract), and unfolding of local functions is applied to the resulting entry-points. The embedding is recursive and terminates at the root of the hierarchy.
- *Modifiers* are unfolded by substituting the prologue and the epilogue code segments into the functions utilizing them and eventually applying local function unfolding to the result.

After unfolding, functions marked with **`private`** or **`internal`** are removed and only entry-points are left.

Step 2. Detecting Vulnerable External Calls. We define a mechanism for detecting violations of the CEI guideline in

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

unfolded code. The mechanism is based on both syntactic patterns in conjunction with type information; in particular, the types of Solidity expressions are required.

We introduce a limited set of *blacklisted* primitives and language constructs performing external calls:

- *Low-level calls*: expressions of form $e_0.\texttt{call}(e_1)$ where expression $e_0 : \texttt{address}$ and argument $e_1 : \texttt{bytes}$.
- *High-level calls*: expressions of form $e_0.\texttt{f}(e_1, \ ..., \ e_n)$ where $n \geq 0$, e_1 .. e_n are arguments of any type, `f` is a function name identifier, $e_0 : \tau$ is the result of an explicit type-cast (either in-place or earlier in the scope) of form $\tau(e)$, where $e : \texttt{address}$.

The first bullet point detects proper invocations to the `call` primitive. Types are required to rule out invocations of user-defined methods named `call`. The second bullet point detects external calls that exhibit an address type-cast into an interface or contract type name.

For each unfolded entry-point function $f'(x_1 : \tau_1, .., x_n : \tau_n)\ \mathcal{S}'$ with $\mathcal{S}' = \{S'_1; ...; S'_k\}$, f' is considered **reentrant** if and only if *both* the following conditions hold:

- $\exists S'_i \in \mathcal{S}'$, for some $i \in [1, k]$, performing an *external call* belonging to the set of blacklisted calls;
- **and** $\exists S'_j \in \mathcal{S}'$ such that $j > i$ performing some *side effect* on a state variable.

The following statements are considered **side effects**:

- the assignment statement $e_1 = e_2$, where e_2 is an expression of any form and e_1 belongs to the following forms:
  - plain identifiers of form x that belong to the contract state variables (local variables are ignored);
  - field select $e.x$, where expression $e : \tau$ such that τ is a contract or struct type and x is a field of τ;
  - array subscript $e[e']$, where $e : \tau[\,]$ and $e' : \texttt{uint}$;
- statements including a subexpression of form $e_0.\texttt{delegatecall}(e_1)$ where expression $e_0 : \texttt{address}$ and argument $e_1 : \texttt{bytes}$.

Event emission (`emit`) and other special statements such as `require`, `assert`, `error`, and `revert` are *not* considered side effects. When Step 2 detects a side effect after an external call, the contract is marked as *candidate* reentrant and Step 3 is necessary to rule out false positives; otherwise it is marked as *safe* and the procedure stops.

Step 3. Contract State Comparison.

We introduce the notion of *state of a contract* as the tuple of state variables, i.e., global fields therein defined, plus the state of the contract internal balance. Such state can either be valid or invalid due to an error ($\perp$). By error, we mean a transaction revert, whether deliberately triggered by the user code or induced by the EVM during execution.

Let σ_0 denote the contract state prior to the invocation of a candidate reentrant function f, and let σ_n denote the state resulting from n reentrant invocations of f within a single transaction. Similarly, let σ'_n denote the state produced by n independent non-reentrant calls to f, each executed in separate transactions. A contract is vulnerable to reentrancy if and only if $\sigma_n \neq \sigma'_n$, meaning that chaining through reentrant calls induces an observable divergence in the state that would not arise under isolated executions. Error states make no exception: when both executions terminate in error, i.e. when $\sigma_n = \sigma'_n = \perp$, the contract is deemed safe. Conversely, if either σ_n or σ'_n becomes unreachable due to an intermediate error, i.e. a state transition $\sigma_i = \perp$ or $\sigma'_i = \perp$ for some $i < n$, then the contract is classified as reentrant. To be considered safe, a contract must therefore reach the same state, whether valid or not, in the same number of steps in both forms, i.e. as a single transaction performing reentrant calls and as separate sequential transactions.

While the procedure outlined above is presented at a semi-formal level, it is designed as a reproducible workflow for manual labelling by expert human analysts. For sufficiently simple programs with a manageable number of external calls, each step is practical to perform, including step 3, despite its semantic nature. A complete example of application of the procedure is available as supplementary material.

V. Methodology

In this section we delve into the details of how we run the tool suite presented in Table I on our two datasets, and how we eventually extracted the relevant information from their output logs in order to compute the proper metrics.

A. Experimental Setup

Experiments utilized the two verified datasets: the Aggregated Benchmark and the Reentrancy Scenarios Dataset (RSD). For tool analysis, we adopted tools from Table I, and used SmartBugs 2.0 [27] to collect the results needed to assess reentrancy detection performance.

All non-LLM models were evaluated through 3-fold cross-validation. Traditional ML models used `scikit-learn` v1.6.0 with TF-IDF features (max 2048) derived from Solidity code, optimizing hyperparameters via a grid search. DL models were implemented with `PyTorch` v2.4.1 and `transformers` v4.47.0, including a BiLSTM (hidden size 128) model using CodeBERT embeddings and a fine-tuned version of CodeBERT (`microsoft/codebert-base`), trained for up to 20 epochs with a learning rate of 1×10^{-3} and a batch size of 32, guided by a fixed validation set for early stopping. For the RSD, each training set fold was augmented with the entire Aggregated Benchmark to improve learning.

LLMs were evaluated zero-shot with a temperature of 0 to ensure deterministic outputs and reproducibility across runs, following empirical evaluations of LLM-based program analysis [46]. Beyond specifying the output format, the prompt represents a less formal version of the labeling procedure described in Section IV-C, defining external calls, state modifications, and reentrancy guards, and guiding the model to recognize complex scenarios such as cross-function and cross-contract reentrancy. Full prompt structures are available in our code repository.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Model Performance Metrics on two datasets: *Aggregated Benchmark* and *RSD*.

		Aggregated Benchmark					*Reentrancy Scenarios Dataset (RSD)*				
	Tool/Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**Error (%)**	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**Error (%)**
Analyzers	Aderyn	-	-	-	-	100.00	0.66	0.62	0.79	0.70	0.00
	CCC	0.89	0.91	0.66	0.76	25.46	0.58	0.74	0.24	0.36	0.00
	Confuzzius	0.88	0.95	0.62	0.75	15.97	0.58	0.57	0.61	0.59	12.59
	Conkas	0.84	0.67	0.78	0.72	18.18	-	-	-	-	100.00
	Mythril	0.86	0.84	0.48	0.61	7.22	0.67	0.67	0.66	0.67	0.00
	Oyente+	0.91	0.92	0.75	0.83	0.00	0.60	0.72	0.32	0.45	0.00
	Sailfish	0.83	0.93	0.42	0.58	0.00	-	-	-	-	100.00
	Securify	0.88	**1.00**	0.48	0.65	25.54	-	-	-	-	100.00
	Sfuzz	0.76	0.71	0.24	0.36	45.14	-	-	-	-	100.00
	Slither	0.95	0.88	0.95	0.92	0.00	0.74	0.71	0.82	0.76	0.00
	Solhint	0.81	0.90	0.36	0.51	0.69	0.55	0.59	0.28	0.38	0.00
	Vandal	0.74	0.66	0.52	0.58	68.72	0.52	0.57	0.25	0.34	78.24
Machine Learning	gradient_boosting	0.90 ± 0.04	0.87 ± 0.04	0.76 ± 0.11	0.81 ± 0.05	-	0.62 ± 0.02	0.63 ± 0.03	0.62 ± 0.02	0.61 ± 0.02	-
	gnb	0.82 ± 0.05	0.70 ± 0.06	0.65 ± 0.12	0.67 ± 0.07	-	0.58 ± 0.03	0.71 ± 0.05	0.58 ± 0.03	0.51 ± 0.07	-
	knn	0.88 ± 0.01	0.85 ± 0.13	0.74 ± 0.13	0.78 ± 0.01	-	0.62 ± 0.01	0.62 ± 0.01	0.62 ± 0.01	0.62 ± 0.01	-
	logistic_regression	0.80 ± 0.06	0.93 ± 0.12	0.33 ± 0.12	0.47 ± 0.12	-	0.49 ± 0.01	0.33 ± 0.12	0.49 ± 0.01	0.35 ± 0.03	-
	random_forest	0.90 ± 0.04	0.90 ± 0.07	0.73 ± 0.13	0.80 ± 0.07	-	0.63 ± 0.03	0.64 ± 0.03	0.63 ± 0.03	0.62 ± 0.04	-
	svm	0.84 ± 0.04	0.93 ± 0.12	0.51 ± 0.16	0.64 ± 0.09	-	0.50 ± 0.00	0.33 ± 0.01	0.50 ± 0.00	0.34 ± 0.02	-
	xgboost	0.91 ± 0.02	0.88 ± 0.08	0.79 ± 0.10	0.83 ± 0.02	-	0.72 ± 0.03	0.72 ± 0.03	0.72 ± 0.03	0.72 ± 0.03	-
	codebert	0.90 ± 0.13	0.82 ± 0.24	$\mathbf{0.96} \pm 0.03$	0.87 ± 0.14	-	0.65 ± 0.06	0.67 ± 0.07	0.65 ± 0.06	0.64 ± 0.06	-
	lstm	0.86 ± 0.12	0.76 ± 0.17	$\mathbf{0.96} \pm 0.03$	0.83 ± 0.11	-	0.60 ± 0.05	0.62 ± 0.06	0.60 ± 0.05	0.59 ± 0.05	-
	ffnn	$\mathbf{0.96} \pm 0.01$	0.85 ± 0.03	0.89 ± 0.03	0.86 ± 0.02	-	0.61 ± 0.03	0.62 ± 0.03	0.61 ± 0.03	0.61 ± 0.03	-
LLMs	gemini-2.0-flash	0.87	0.87	0.84	0.85	-	0.51	0.52	0.52	0.49	-
	gemini-2.5-flash	0.83	0.85	0.82	0.83	-	0.58	0.66	0.58	0.53	-
	gpt-4o	0.85	0.89	0.85	0.86	-	0.53	0.53	0.53	0.51	-
	gpt-4.1	0.87	0.90	0.87	0.88	-	0.63	0.63	0.63	0.63	-
	o3-mini	**0.96**	0.96	**0.96**	**0.96**	-	0.79	0.80	0.79	0.79	-
	o4-mini	0.94	0.94	0.94	0.93	-	0.82	0.83	**0.82**	**0.82**	-
	gpt-oss	0.94	0.96	0.82	0.88	-	**0.87**	**0.98**	0.75	0.85	-
	qwen3	0.91	0.78	0.93	0.85	-	0.68	0.66	0.73	0.69	-
	phi4	0.80	0.58	**0.96**	0.72	-	0.56	0.55	0.65	0.60	-

Commercial LLMs were queried via vendor APIs from April 1 to May 20, 2025. Model snapshots used include `gpt-4o-2024-08-06`, `gpt-4.1-2025-04-14`, `o3-mini-2025-01-31`, `o4-mini-2025-04-16`; `gemini-2.0-flash-001`, and `gemini-2.5-flash-preview-05-20`. Explanations from `o3-mini` for 146 contracts in the Aggregated Benchmark were rated by three of the authors in a within-subject, randomized design, on a 5-point Likert scale. This evaluation was complemented by `o4-mini`'s analysis of all `o3-mini` explanations.

Alongside commercial LLMs, we experimented with open-source models queried through their latest snapshots (December 2025) pulled from the Ollama repositories. A preliminary analysis of model language generation performance by size showed that small models (<10B parameters) were inadequate for the task at hand. Namely, we experimented with Gemma3:4b and Llama3.1:8b: they both struggled to generate structured output in a parsable JSON format, and even when they succeeded, their classification accuracy was underwhelming (with a strong bias towards classifying all contracts as safe). Thus, we shifted our focus towards the largest models that could fit consumer hardware (all local experiments were conducted on an NVIDIA GTX 5080 GPU). We selected three state-of-the-art mid-size LLMs for which we report accuracy performance: gpt-oss:20b, qwen3:14b, phi4:14b.

B. Evaluation metrics

The results derived from our two datasets (Table IV) highlight distinct capabilities with important implications for reentrancy detection. For each row, we report standard metrics such as accuracy and F1 score, as well as precision and recall, since these represent two critical aspects of our analysis. Low precision indicates a high number of false positives, a common criticism of static and formal-methods-based tools. Low recall, on the other hand, corresponds to many false negatives, which detectors should minimize. As the last metric, we report the error rate. For each tool and each contract, the detection output is considered an error/fail if the tool exited with an error code and did not provide any findings in the list of supported vulnerabilities. While we initially discarded all outputs with exit codes different from 0, we observed that tool performance decreased drastically, so we applied this mitigation.

VI. Results

We present the results of our experiments on both the Aggregated Benchmark and the RSD in Table IV, divided into horizontal groups. The first group includes detection tools and analyzers based on formal methods; the second group presents machine learning techniques and vanilla networks; the third group gathers commercial LLMs (called by API) as well as models installed locally (separated by a thin horizontal line). For each group and for each column, we underline the best result for the group, and mark the best result overall (of the whole column) in bold. For machine learning techniques, we also report the standard deviation, while the error is missing from both the machine learning and LLMs categories because it does not apply (the models always provide an answer).

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

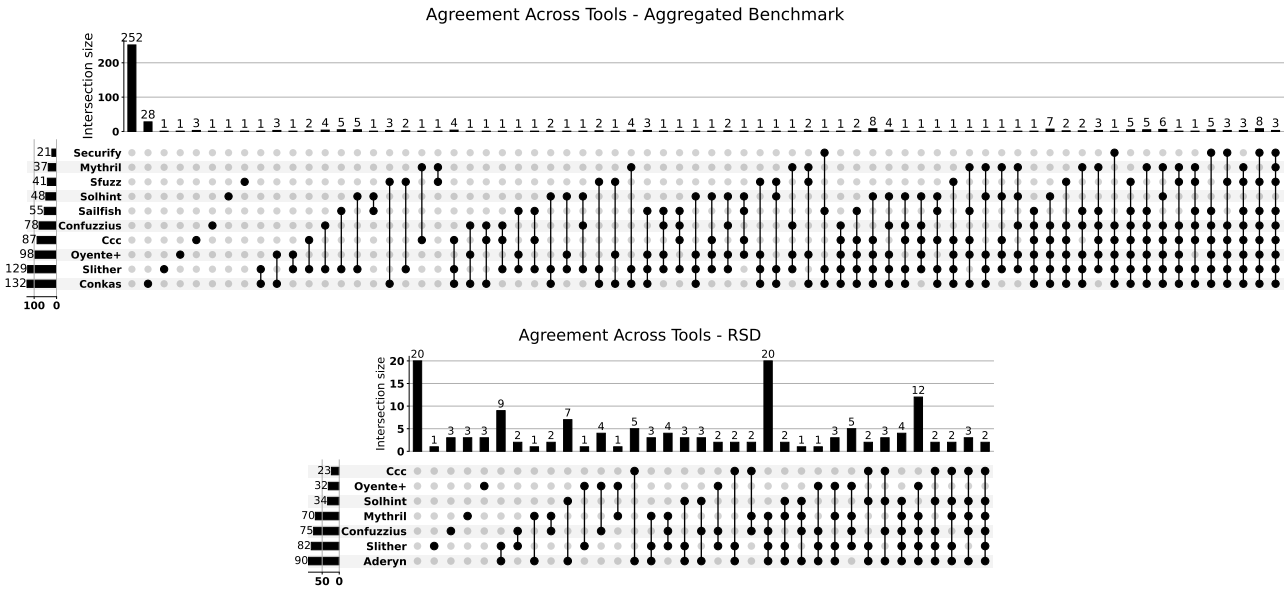


Fig. 1: Comparison between upset plots for Aggregated Benchmark and RSD datasets. This type of plot serves as an alternative to Venn diagrams for visualizing intersections and overlaps among more than three sets. The histograms indicate the number of samples per row and column, while the dots show which tools flagged the contracts within each set.

*A. **RQ1:** How reliable and robust are existing reentrancy detection tools when applied to modern Solidity codebases and evaluated under stress-testing scenarios, given their lack of maintenance and outdated compiler support?*

To answer this, we analyze the error rates in Table IV. Of the 11 tools running on the Aggregated Benchmark, only 7 can process RSD contracts. Of these, just 5 complete the analysis without failures. A notable exception is Aderyn, a recent tool that focuses exclusively on newer Solidity versions. CCC and Oyente+ also show broader version support. Across the RSD, tools tend to either fully support or entirely fail on contracts (with Confuzzius and Vandal being the only partial cases).

Older tools that remain actively maintained, such as Mythril, Slither, and Solhint, still encounter a substantial number of errors even in the Aggregated Benchmark. This can be attributed to the wide range of compiler versions in the dataset and the numerous syntax and semantic changes introduced within Solidity 0.4.x and 0.5.x. Moreover, most analyzers operate at the bytecode (EVM opcode) level, where historical differences in EVM revisions further complicate compatibility. High error rates also influence the interpretation of performance metrics. For instance, Securify reports 100% precision in the RSD dataset primarily because many contracts it fails to analyze are labeled as reentrant, inflating the metric. Overall, tool performance degrades noticeably when moving from the Aggregated Benchmark to the RSD, even when considering only analyzers that do not report errors.

*B. **RQ2:** How consistent are the warnings and vulnerability reports produced by different tools when applied to the same contracts?*

Figure 1 shows two upset plots illustrating how much different tools agree when reporting reentrancy vulnerabilities. Each plot can be read as follows: the horizontal bars on the left indicate how many contracts each tool flagged, while the filled dots below the plot identify which tools are included in each intersection. A vertical bar is then drawn above every connected group of dots, quantifying how many contracts were jointly marked as reentrant by that specific combination of tools. Thus, the dots define the tool combination and the height of the bar shows the size of the agreement. The plots reveal very limited agreement in real-world scenarios (Aggregated Benchmark) and in MWEs (RSD).

For the top plot, all tools agree on the lack of reentrancy over 252 contracts, regardless of the ground truth. On the opposite side, though, if we consider the agreement among at least 6 tools, we count only 78 contracts out of 122 labeled as reentrant. For the RSD the agreement is a bit stronger, probably due to the reduced number of tools supporting Solidity v0.8 and the dataset size. In particular, there is a similarity in the detection of Mythril, Confuzzius, Slither, and Aderyn (20 contracts). These results suggest automated tools tend to converge only when reentrancy cases are simple and well-defined, while more realistic implementations still lead to substantial disagreement across analyzers.

For completeness, we report which RSD categories are most frequently misclassified by the four top-performing tools:

	Code	Variants
Aderyn	00, 03, 04, 12, 14	Fold, Inline, Underflow, Mod
Mythril	00, 01, 04, 09, 12, 13, 14, 15	Error, Mod, Fold, Staticcall
Oyente+	00, 01, 02, 03, 04, 07, 08, 09, 13, 14, 15	Inline, Fold, Underflow, Unchecked, Emit, Mod, Staticcall
Slither	00, 09, 14, 15	Inline, Mod, Staticcall

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

Oyente+ misses several cases due to unsupported post-0.8 features, while Mythril and Slither overlook some baseline scenarios (code 00, see Table III and Section IV-B) depending on how CEI-breaking is interpreted (e.g., whether `error` and `emit` count as effects). Static calls are often deemed harmless, whereas our procedure labels them as exploitable via read-only reentrancy [51]. Aderyn, operating on source code, misses subtler patterns such as function folding and nested modifiers; cross-function cases with mutex-style guards (code 04) are also challenging. Delegatecalls (code 14) are not always conservatively treated as malicious. Overall, discrepancies persist due to divergent reentrancy interpretations and mismatches between published descriptions and actual implementations.

C. ***RQ3:*** *Do general-purpose Large Language Models (LLMs) demonstrate higher dependability and lower maintenance cost compared to static analysis and ML-based detectors?*

As shown in Table IV, LLMs achieve the best overall performance across both datasets. In the Aggregated Benchmark, `o3-mini` nearly reaches perfect accuracy, surpassing both the strongest static analyzer (Slither) and the top ML model (FFNN). In the RSD dataset, `gpt-oss` achieves the highest scores among all evaluated approaches, confirming that high accuracy is attainable even with fully local models.

LLMs also provide stronger robustness: unlike static analyzers, they do not fail due to unsupported compiler versions or outdated vulnerability rules, resulting in zero reported errors in both datasets. This reliability stems from their ability to reason directly over source semantics rather than relying on hand-crafted bytecode patterns.

In addition, maintenance costs are negligible: improvements in foundation models generally improve detection performance without requiring rule updates or model retraining, although newer models may not outperform earlier ones in all cases. Notably, the commercial LLMs evaluated in this study (GPT-4.1, GPT-4o, o4-mini) have already been superseded by the more capable GPT-5.1 family. Our results therefore represent a conservative lower bound on current LLM performance, further reinforcing the widening robustness gap relative to static analyzers.

Overall, LLMs introduce a tradeoff between determinism and robustness: traditional analyzers provide reproducible rule-based reasoning but require continuous maintenance, whereas LLMs sacrifice strict determinism while remaining robust to language evolution and unseen attack patterns. As foundation models improve without manual updates, this tradeoff is likely to increasingly favor LLM-based approaches.

D. ***RQ4:*** *Can LLM-generated explanations assist human experts in identifying mislabeled or ambiguous cases, thus improving dataset quality?*

LLMs played a central role in validating the annotations of our two datasets. Traditional analysis tools often produce logs that are difficult to interpret, while LLMs return concise and readable explanations that identify the vulnerable function and the relevant code location. This clear communication is particularly important in security analysis, where human oversight is essential for trust. During our verification process, reviewing the explanations not only confirmed detection outcomes but also revealed multiple annotation issues.

In particular, LLM-generated explanations allowed us to identify 9 contracts in the Aggregated dataset that were initially mislabeled due to subtle or ambiguous reentrancy behavior. By comparison, Slither detected only 3 mislabeled cases in the RSD dataset, and understanding its reasoning required significantly more time than analyzing the explanations provided by `o3-mini`. This demonstrates the practical value of LLMs in iterative dataset improvement and supports their use as complementary tools for security auditing.

After dataset construction, the three authors evaluated the goodness of the explanation for contracts misclassified by `o3-mini`. Given the complexity and the large number of explanations to evaluate (more than $560\,000$, considering all models and all contracts), we used an LLM-as-a-judge approach for the rest of the responses. We report inter-rater reliability using α (Krippendorff's Alpha [37]) for ordinal Likert scores, which accounts for chance agreement and penalizes larger disagreements more heavily. This allows comparing both human consistency and alignment between humans and the LLM.

Metric	Human Mean (n=3) (Misclassified)	LLM Mean (n=1) (Full Set)	Inter-Human α	Human-LLM α
Correctness	4.38 ± 0.94	4.46 ± 0.91	0.84	0.72
Informativeness	4.14 ± 0.70	3.32 ± 0.81	0.79	0.41
Pertinence	4.37 ± 0.87	4.81 ± 0.57	0.81	0.58

Human reviewers scored explanations based on correctness, informativeness, and pertinence, showing generally positive results with high inter-human agreement across all dimensions ($\alpha \in [0.79, 0.84]$), confirming that the evaluation task is well-defined. The automatic evaluation over the full dataset yields comparable correctness and higher pertinence, but lower informativeness. Comparing inter-human and human–LLM agreement reveals a reliability gap. The LLM aligns closely with human judgments on correctness ($\alpha = 0.72$), but shows substantially weaker agreement on informativeness ($\alpha = 0.41$) and moderate alignment on pertinence ($\alpha = 0.58$). This suggests that while the model reliably captures objective properties, it diverges from human expectations on more subjective dimensions. In particular, the lower agreement and mean score for informativeness indicate stricter or misaligned evaluation criteria. Qualitative feedback indicates that the most effective explanations pointed to specific functions or critical state changes within the code, while a minority of explanations lacked useful detail.

Overall, these observations show that LLMs offer more helpful and actionable insights than current static analyzers and can support human experts in verifying challenging examples. Their involvement leads to improved dataset reliability and contributes to more effective benchmarking in smart contract security.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

VII. Threats to Validity

Internal validity. Manual relabeling may still contain residual errors, despite three-reviewer consensus and LLM-assisted cross-checks. However, our results show a *systematic lack of agreement* among tools themselves (Table IV, Figure 1), including clear-cut cases. These divergences persist even if annotation noise is assumed, indicating that detector inconsistencies stem primarily from tool heuristics rather than ground truth quality. Execution outcomes also depend on the SmartBugs framework 2.0.15 used to run most tools, and on legacy compiler support when analyzing modern Solidity code. We mitigated instability by executing tools in a controlled environment and repeating failing runs, but partial outputs may still influence individual results.

Additionally, using LLMs to assist humans in the manual labeling process may introduce subtle confirmation or anchoring biases, potentially influencing how borderline or ambiguous cases are interpreted. If the labeling process is partially shaped by model-generated explanations or suggestions, this could inadvertently align the ground truth with patterns more easily recognized by similar models. Such an effect may partly explain why `o3-mini` achieves particularly high performance in our evaluation.

External validity. The Aggregated Benchmark includes real-world contracts but is not statistically representative of the entire Ethereum ecosystem, while the RSD intentionally focuses on semantic stress tests rather than prevalence. Thus, our findings should not be interpreted as estimates of ecosystem-wide vulnerability rates; rather, they provide evidence of reliability and robustness gaps in current detection tools. Broader studies will be needed to assess how these conclusions extend across more diverse contract populations and execution environments.

VIII. Related Work

Empirical evaluations of smart contract analyzers commonly rely on manually labeled datasets spanning multiple vulnerability classes. Frameworks such as SmartBugs [33] and subsequent curated benchmarks evaluate static and dynamic analyzers using heterogeneous vulnerability corpora, while recent studies provide large manually annotated datasets with line-level labels for several vulnerability types [52]. Although these efforts improve benchmarking reproducibility, reentrancy represents only one category among many, resulting in limited semantic coverage of this specific vulnerability, especially when vulnerabilities are injected [36, 18].

A second line of work focuses explicitly on reentrancy detection. Prior studies and tools analyze reentrancy patterns or propose specialized detection techniques [68, 35, 64]. However, these works typically consider a restricted set of analyzers and do not explicitly characterize *dependability under language evolution*, i.e. whether detectors remain functional and reliable on modern Solidity (0.8+), where syntax and semantics have substantially changed. Moreover, existing reentrancy-focused benchmarks do not combine *minimal working examples* designed as controlled stress tests that isolate specific semantic corner cases (e.g., `delegatecall`, `staticcall`, proxies, folded calls) with a fine-grained taxonomy that goes beyond the traditional macroscopic categories (single-function, cross-function, cross-contract etc.), making it difficult to attribute failures to precise reentrancy mechanisms rather than to incidental contract complexity.

More recently, several works have explored the use of large language models for smart contract vulnerability detection [20, 24, 62, 54]. These studies demonstrate promising capabilities of LLMs but evaluate multiple vulnerability classes simultaneously, without conducting a dedicated analysis of reentrancy semantics or robustness across evolving language features.

In contrast, the present work concentrates exclusively on reentrancy and provides a unified dependability evaluation across static analyzers, ML-based approaches, and modern LLMs, grounded in two manually verified datasets designed to capture both real-world contracts and semantically challenging reentrancy scenarios.

IX. Conclusion

This work revisits the state of reentrancy detection from a dependability perspective. We introduce two manually verified datasets: the *Aggregated Benchmark*, built from three prior human-labeled corpora, and the *Reentrancy Scenarios Dataset (RSD)*, a stress-testing suite covering modern Solidity features and semantic corner cases. Using these datasets, we evaluate 12 traditional tools, 10 ML/DL models, and 9 LLMs.

Our findings expose a fractured and unreliable ecosystem. Despite strong F1 scores for detectors such as Oyente+ (83%) and Slither (92%), the remaining tools perform extremely poorly, often failing with errors or producing no meaningful output. More concerning is the lack of consistency: on the Aggregated Benchmark, only 38 out of 122 reentrant contracts exhibit agreement among at least 6 tools, meaning that even a minimal majority consensus is reached for barely one third of the cases. On the RSD, many tools fail outright due to outdated compiler support or simplistic syntactic heuristics. In contrast, multiple LLMs achieve above 85% accuracy in zero-shot settings, outperforming all conventional detectors while requiring no maintenance. Their explanations also helped identify mislabeled cases, directly improving dataset reliability.

These results indicate a fundamental shift: as Solidity evolves, static tools are no longer dependable ground truth for security pipelines, while LLMs already provide more robust and semantically grounded analysis. Still, several challenges remain. Our datasets, though carefully curated, cover a limited fraction of deployed contract diversity. Scaling human-in-the-loop validation of LLM explanations and investigating more advanced prompting or fine-tuning strategies represent promising directions. Moreover, hybrid approaches that combine the formal guarantees of static analysis with the adaptability of LLMs may offer the best path toward dependable vulnerability detection in rapidly evolving smart contract ecosystems.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

Data Availability Statement

The data analyzed and discussed in this paper are openly available in a repository together with the supplementary material. The link to the repository is: https://github.com/matteo-rizzo/reentrancy-detection-benchmarks.

Acknowledgments

This paper was funded by Veneto Agricoltura within the scope of the project "Guaranteeing the continuity of the agri-foodchain: the digitization of wholesale markets". This study was also carried out within the PE0000014 - Security and Rights in the CyberSpace (SERICS) and received funding from the European Union Next-GenerationEU - National Recovery and Resilience Plan (NRRP) – MISSION 4 COMPONENT 2, INVESTIMENT 1.3 – CUP N. H73C22000890001. This work has been also partially supported by the Research Project INDAM GNCS 2025 - CUP E53C24001950001 - "Modelli e Analisi per sistemi Reversibili e Quantistici (MARQ)" and by the Project PRIN 2020 - CUP N. 20202FCJMH "NiRvAna - Noninterference and Reversibility Analysis in Private Blockchains". This manuscript reflects only the authors' views and opinions, neither the European Union nor the European Commission can be considered responsible for them.

References

[1] ERC 20 - OpenZeppelin Docs — docs.openzeppelin.com. https://docs.openzeppelin.com/contracts/2.x/api/token/erc20. [Accessed 30-05-2025].

[2] GitHub - christoftorres/confuzzius at v0.0.2 — github.com. https://github.com/christoftorres/ConFuzzius. Accessed November 2025.

[3] GitHub - ConsenSysDiligence/mythril at –v0.24.8 - github.com. https://github.com/ConsenSysDiligence/mythril. Accessed November 2025.

[4] GitHub - ConsenSysDiligence/mythril: Mythril is a symbolic-execution-based securty analysis tool for EVM bytecode. It detects security vulnerabilities in smart contracts built for Ethereum and other EVM-compatible blockchains. — github.com. https://github.com/ConsenSysDiligence/mythril. [Accessed 30-05-2025].

[5] GitHub - crytic/slither at v0.11.3 — github.com. https://github.com/crytic/slither. Accessed November 2025.

[6] GitHub - cyfrin/aderyn at #f4ce952 — github.com. https://github.com/Cyfrin/aderyn. Accessed November 2025.

[7] GitHub - duytai/sfuzz at #ce87440 — github.com. https://github.com/duytai/sFuzz. Accessed November 2025.

[8] GitHub - eth-sri/securify at #51ba124 — github.com. https://github.com/eth-sri/securify. Accessed November 2025.

[9] GitHub - eth-sri/securify2 at #1913dfe — github.com. https://github.com/ethsri/securify2. Accessed November 2025.

[10] GitHub - fraunhofer-aisec/cpg-contract-checker at #4ff0493 — github.com. https://github.com/Fraunhofer-AISEC/cpg-contract-checker. Accessed November 2025.

[11] GitHub - nescio007/teether at #04adf56 — github.com. https://github.com/nescio007/teether. Accessed November 2025.

[12] GitHub - protofire/solhint at v6.0.0 — github.com. https://github.com/protofire/solhint. Accessed November 2025.

[13] GitHub - smartbugs/conkas at #4e0f256 — github.com. https://github.com/smartbugs/conkas. Accessed November 2025.

[14] GitHub - smartbugs/conkas: Ethereum Virtual Machine (EVM) Bytecode or Solidity Smart Contract static analysis tool based on symbolic execution — github.com. https://github.com/smartbugs/conkas. [Accessed 30-05-2025].

[15] GitHub - smartbugs/oyente-plus at v1.0.0 — github.com. https://github.com/smartbugs/oyente_plus. Accessed November 2025.

[16] GitHub - trailofbits/manticore at v0.3.7 — github.com. https://github.com/trailofbits/manticore. Accessed November 2025.

[17] GitHub - usyd-blockchain/vandal at #d2b0043 — github.com. https://github.com/usyd-blockchain/vandal. Accessed November 2025.

[18] GitHub - xf97/HuangGai at v1.0.0 — github.com. https://github.com/xf97/HuangGai/tree/v1.0.0. [Accessed 29-05-2025].

[19] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. A survey of attacks on ethereum smart contracts (SoK). In *Principles of Security and Trust (POST)*, volume 10204 of *LNCS*, pages 164–186. Springer, 2017.

[20] Biagio Boi, Christian Esposito, and Sokjoon Lee. Smart contract vulnerability detection: The role of large language model (llm). *ACM SIGAPP applied computing review*, 24(2):19–29, 2024.

[21] Priyanka Bose, Dipanjan Das, Yanju Chen, Yu Feng, Christopher Kruegel, and Giovanni Vigna. Sailfish: Vetting smart contract state-inconsistency bugs in seconds. In *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, page –. Cornell University, May 2021. arXiv preprint arXiv:2104.08638, accessed https://arxiv.org/abs/2104.08638.

[22] Lexi Brent, Anton Jurisevic, Michael Kong, Eric Liu, François Gauthier, Vincent Gramoli, Ralph Holz, and Bernhard Scholz. Vandal: A scalable security analysis framework for smart contracts. *arXiv preprint arXiv:1809.03981*, 9 2018. Preprint; see project page for code and artifacts.

[23] R. M. Burstall and John Darlington. A transformation system for developing recursive programs. *Journal of the ACM*, 24(1):44–67, 1 1977.

[24] Chong Chen, Jianzhong Su, Jiachi Chen, Yanlin Wang, Tingting Bi, Jianxing Yu, Yanli Wang, Xingwei Lin, Ting Chen, and Zibin Zheng. When chatgpt meets smart contract vulnerability detection: How far are we? *ACM Transactions on Software Engineering and Methodology*,

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

34(4):1–30, 2025.

[25] Jiachi Chen, Xin Xia, David Lo, John Grundy, Xiapu Luo, and Ting Chen. Defectchecker: Automated smart contract defect detection by analyzing evm bytecode. *IEEE Transactions on Software Engineering*, 48(7):2189–2207, 7 2022.

[26] Jaeseung Choi, Doyeon Kim, Soomin Kim, Gustavo Grieco, Alex Groce, and Sang Kil Cha. Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 227–239. IEEE, 2021.

[27] Monika di Angelo, Thomas Durieux, João F. Ferreira, and Gernot Salzer. SmartBugs 2.0: An execution framework for weakness detection in ethereum smart contracts. In *38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 2102–2105. IEEE Computer Society, 9 2023.

[28] Monika Di Angelo and Gernot Salzer. *Consolidation of Ground Truth Sets for Weakness Detection in Smart Contracts*, volume 13953, pages 439–455. Springer Science+Business Media, 12 2023.

[29] Hao Ding, Yizhou Liu, Xuefeng Piao, Huihui Song, and Zhenzhou Ji. Smartguard: An llm-enhanced framework for smart contract vulnerability detection. *Expert Systems with Applications*, 269:126479, 2025.

[30] Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In Gregg Rothermel and Doo-Hwan Bae, editors, *Proceedings of the ACM/IEEE 42nd International conference on software engineering*, pages 530–541. ACM, 6 2020.

[31] Rainer Feichtinger, Robin Fritsch, Lioba Heimbach, Yann Vonlanthen, and Roger Wattenhofer. Sok: Attacks on daos. *arXiv preprint arXiv:2406.15071*, 2024.

[32] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: a static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 8–15. IEEE, IEEE / ACM, 5 2019.

[33] João F Ferreira, Pedro Cruz, Thomas Durieux, and Rui Abreu. Smartbugs: A framework to analyze solidity smart contracts. In *Proceedings of the 35th IEEE/ACM international conference on automated software engineering*, pages 1349–1352. Cornell University, 7 2020.

[34] Christof Ferreira Torres, Mathis Baden, Robert Norvill, Beltran Borja Fiz Pontiveros, Hugo Jonker, and Sjouke Mauw. Ægis: Shielding vulnerable smart contracts against attacks. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, pages 584–597, 2020.

[35] Jun Gao, Yu Fu, Zhi Chen, Defang Ye, Peng Qian, and Kun Sun. Silence false alarms: Identifying anti-reentrancy patterns on Ethereum to refine smart contract reentrancy detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[36] Asem Ghaleb and Karthik Pattabiraman. How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In Sarfraz Khurshid and Corina S. Pasareanu, editors, *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, pages 415–427. ACM, 7 2020.

[37] Andrew F Hayes and Klaus Krippendorff. Answering the call for a standard reliability measure for coding data. *Communication methods and measures*, 1(1):77–89, 2007.

[38] Sukrit Kalra, Seep Goel, Mohan Dhawan, and Subodh Sharma. Zeus: analyzing safety of smart contracts. In *Ndss*, pages 1–12, 2018.

[39] Johannes Krupp and Christian Rossow. teether: Gnawing at ethereum to automatically exploit smart contracts. In William Enck and Adrienne Porter Felt, editors, *27th USENIX Security Symposium (USENIX Security '18)*, pages 1317–1333, Baltimore, MD, USA, August 2018. USENIX Association.

[40] Yuqi Liu, Rui Xi, and Karthik Pattabiraman. Reentrancy redux: The evolution of real-world reentrancy attacks on blockchains. In *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 576–588. IEEE, 2025.

[41] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 254–269. ACM, 10 2016.

[42] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 254–269. ACM, 10 2016.

[43] Deepak Kumar Mishra and Pawan Singh Mehra. Totalsol: A multi-layer static analysis method for vulnerability detection in ethereum based smart contracts. In *2025 International Conference on Cognitive Computing in Engineering, Communications, Sciences and Biomedical Health Informatics (IC3ECSBHI)*, pages 945–950. IEEE, 1 2025.

[44] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. Manticore: A user-friendly symbolic execution framework for binaries and smart contracts, 11 2019.

[45] Tai D Nguyen, Long H Pham, Jun Sun, Yun Lin, and Quang Tran Minh. sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In Gregg Rothermel and Doo-Hwan Bae, editors, *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*,

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

volume abs/2004.08563, pages 778–788. Cornell University, 4 2020.

[46] Shuyin Ouyang, Jie M Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of chatgpt in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28, 2025.

[47] Helmuth Partsch and Ralf Steinbrüggen. Program transformation systems. *ACM Computing Surveys (CSUR)*, 15(3):199–236, 9 1983.

[48] Daniel Perez and Benjamin Livshits. Smart contract vulnerabilities: Vulnerable does not imply exploited. In Michael Bailey and Rachel Greenstadt, editors, *30th USENIX Security Symposium (USENIX Security 21)*, pages 1325–1341. USENIX Association, 2021.

[49] Dalila Ressi, Alvise Spanò, Lorenzo Benetollo, Carla Piazza, Michele Bugliesi, and Sabina Rossi. Vulnerability detection in ethereum smart contracts via machine learning: A qualitative analysis. *arXiv preprint arXiv:2407.18639*, abs/2407.18639, 7 2024.

[50] Michael Rodler, Wenting Li, Ghassan O. Karame, and Lucas Davi. Sereum: Protecting existing smart contracts against re-entrancy attacks. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2019. Runtime defense and taxonomy of advanced re-entrancy patterns; shows attacks beyond the DAO archetype, including those using method calls.

[51] Ioannis Sachinoglou. Read-only reentrancy. ChainSecurity presentation/pdf, 2023. "A novel attack class responsible for $100M+ funds at risk".

[52] Francesco Salzano, Cosmo Kevin Antenucci, Simone Scalabrino, Giovanni Rosa, Rocco Oliveto, and Remo Pareschi. An empirical analysis of vulnerability detection tools for solidity smart contracts using line level manually annotated vulnerabilities. *arXiv preprint arXiv:2505.15756*, 2025.

[53] Clara Schneidewind, Ilya Grishchenko, Markus Scherer, and Matteo Maffei. ethor: Practical and provably sound static analysis of ethereum smart contracts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 621–640. ACM, 10 2020.

[54] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.

[55] Sergey Tikhomirov, Sergey Tikhomirov, and Alexey Frolov. Smartcheck: Static analysis of solidity smart contracts. In *arXiv preprint arXiv:1807.03325*, 2018.

[56] Christof Ferreira Torres, Antonio Ken Iannillo, Arthur Gervais, and Radu State. Confuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 103–119. IEEE, IEEE, 9 2021.

[57] Petar Tsankov, Andrei Dan, Dana Drachsler-Cohen, Arthur Gervais, Florian Buenzli, and Martin Vechev. Securify: Practical security analysis of smart contracts. In David Lie, Mohammad Mannan, Michael Backes, and Xiaofeng Wang, editors, *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, volume abs/1806.01143, pages 67–82. Cornell University, 6 2018.

[58] Valentin F Turchin. The concept of a supercompiler. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 8(3):292–325, 6 1986.

[59] Nuno Veloso. Conkas: A modular and static analysis tool for ethereum smart contracts. In *Master's Thesis, Instituto Superior Tecnico, Universidade de Lisboa*, 2023. Available as technical report via https://fenix.tecnico.ulisboa.pt/downloadFile/1689244997262417/94080-Nuno-Veloso_resumo.pdf.

[60] Shuai Wang, Chengyu Zhang, and Zhendong Su. Detecting nondeterministic payment bugs in ethereum smart contracts. *Proc. ACM Program. Lang.*, 3(OOPSLA):1–29, October 2019.

[61] Wei Wang, Jingjing Song, Guangquan Xu, Yidong Li, Hao Wang, and Chunhua Su. Contractward: Automated vulnerability detection models for ethereum smart contracts. *IEEE Transactions on Network Science and Engineering*, 8(2):1133–1144, 1 2020.

[62] Zhiyuan Wei, Jing Sun, Yuqiang Sun, Ye Liu, Daoyuan Wu, Zijian Zhang, Xianhao Zhang, Meng Li, Yang Liu, Chunmiao Li, et al. Advanced smart contract vulnerability detection via llm-powered multi-agent systems. *IEEE Transactions on Software Engineering*, 2025.

[63] Konrad Weiss, Christof Ferreira Torres, and Florian Wendland. Analyzing the impact of copying-and-pasting vulnerable solidity code snippets from question-and-answer websites. *arXiv preprint arXiv:2409.07586*, pages 713–730, 11 2024. Preprint; contains links to tools CCC and CCD.

[64] Yin Wu, Xiaofei Xie, Chenyang Peng, Dijun Liu, Hao Wu, Ming Fan, Ting Liu, and Haijun Wang. Advscanner: Generating adversarial smart contracts to exploit reentrancy vulnerabilities using llm and static analysis. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1019–1031, 2024.

[65] Shuo Yang, Jiachi Chen, Mingyuan Huang, Zibin Zheng, and Yuan Huang. Uncover the premeditated attacks: Detecting exploitable reentrancy vulnerabilities by identifying attacker contracts. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–12. ACM, 4 2024.

[66] Jingwen Zhang, Zibin Zheng, Yuhong Nan, Mingxi Ye, Kaiwen Ning, Yu Zhang, and Weizhe Zhang. Smartreco: Detecting read-only reentrancy via fine-grained cross-dapp analysis. *arXiv preprint arXiv:2409.18468*, pages 2138–2150, 9 2024.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.

[67] Mengya Zhang, Xiaokuan Zhang, Yinqian Zhang, and Zhiqiang Lin. TXSPECTOR: Uncovering attacks in ethereum from transactions. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2775–2792. USENIX Association, August 2020.

[68] Zibin Zheng, Neng Zhang, Jianzhong Su, Zhijie Zhong, Mingxi Ye, and Jiachi Chen. Turn the rudder: A beacon of reentrancy detection for smart contracts on ethereum. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, page 295–306. IEEE Press, 5 2023.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:30:46 UTC from IEEE Xplore. Restrictions apply.